

ITE 1942 - ICT PROJECT
PROJECT REPORT
Level 01

Pharmacy Inventory Management System
for E Medicare pvt ltd

Submitted by:
Athapaththu DAMPM
E2240344

Bachelor of Information Technology (External Degree)
Faculty of Information Technology
University of Moratuwa

Table of Contents

Chapter 01 Introduction.....	4
1.1Background & Motivation	4
1.2Problem in brief	5
1.3Aim and Objectives.....	5
Aim	5
Objectives	5
Chapter 02 Related Work	6
2.1Introduction.....	6
2.2Similar systems.....	6
Chapter 03 Systems Analysis	9
3.1Introduction.....	9
3.2System requirements	9
staff registration	10
Category Management	10
product management	10
Customer Management	11
Order Management	11
Functional Requirements	11
Non-Functional Requirements.....	11
Chapter 04 System Design.....	12
4.1Staff Registration	13
Category Management	15
Product Management	17
Customer Management	19
order management	21
Pseudocode.....	23
Logging	23
Staff Management.....	23
Category Management	24
Product Management	25
Customer Management	26
Order Management	26

Appendix.....	28
Login interfaces.....	28
Code segments	29
Login Form Code Segment	29
Staff Report Form Code Segment	31
Product Report.....	34
Product Report Form Code Segment.....	34
Form Code Segment	39
References.....	50

Chapter 01 Introduction

In today's scenario, medicines play a crucial role in ensuring the well-being and survival of patients. Procurement of local and foreign drugs, along with various health related equipment is essential for the smooth functioning of pharmacies. This inventory includes a wide range of drugs and brands that cater to the diverse needs of a significant customer base. However, manual management of this complex system proves to be challenging, as extensive paperwork is involved, and employees must devote a lot of time. Storage of day-to-day records requires a large amount of storage capacity, and since anyone can access the system's system security, records can also be missed due to inefficient manual systems. Recognizing these challenges, the firm has decided to streamline its inventory management through the implementation of a computerized system. This automation simplifies tasks for employees, allowing them to navigate through a user-friendly standalone application. This application facilitates searching available items, generating reports, creating invoices and many other functions. The transition to a computerized inventory management system is aimed at increasing the efficiency and effectiveness of the organization's overall operations.

1.1Background & Motivation

After exploring websites like Pharmacy2U, Health Guard Pharmacy, Union Chemist Pharmacy, and Apollo Pharmacy, I am convinced that a technology based computerized inventory management system is very important for organizations like E Medicare. These sites are very user-friendly and offer many features to simplify tasks such as item searches and availability checks. They offer a wide range of medicines and health-related equipment, categorized, and sorted alphabetically for easy access.

Vibrant and colorful interfaces with graphical elements including images make the user experience pleasant and stress-free. The inclusion of search bars and sorting features reduces time consumption and facilitates quick access to specific items. They experience many benefits by providing many features. They provide search bars to view available items, they reduce time consumption, and, in some systems, they provide sorting features to easily access a particular item without any delay. If the E Medicare system is automated by providing such features, the inventory can be handled in a very effective and efficient manner. Furthermore, the organization can ensure the security of important data by providing some encryption features. In addition, it can be ensured that every record is under control, so the chance of missing records is very less, and it is impossible to compare with the existing system.

1.2 Problem in brief

The existing manual system is handling inventory by doing so much paperwork. When someone need to complete his or her task urgently, they need to face lots of difficulties, such as accessing record by finding files, record day to day transaction in manual ways, etc. Prepare bill for customer, staff details handling, record available items, search available categories are done through writing on papers and calculate by manually. It is impossible that generate report and it is harder to manage available items. In addition, employees work without any specification to a specific work, as well as some employees need to do additional works such as printing bills, etc.

It is very annoying thing when employee need to access files because all the files are stored in so many cupboards. Moreover, when calculating bills and transaction it is requires so many times and some calculations can be incorrect. It can be harm to calculate profit and expenses of the organization and it can be harm to good will of the organization. The best solution for those problems is implementing a computerized system, to generate report, bills, and search available items.

1.3 Aim and Objectives

Aim

- The project aims to design and develop computer-aided inventory management and streamline pharmacy operations.

Objectives

- To provide sub system for manage users
- To provide sub system for order items
- To provide sub system for manage categories
- To provide sub system for generate bill
- To provide generate staff report

Summary

The project's origins are explained in the first chapter, which also lists a number of issues with the current system, including challenges with report generation, bill calculation, record access, item availability checks, and so on. Additionally, five works that are linked to the project's topic are explained in the upcoming chapter.

Chapter 02 Related Work

2.1Introduction

After identifying the problems felt by E Medicare Pharmacy, they realized that the best solution was to introduce an inventory management application. E Related to the domain of the Medicare system, there are several types of systems in the world. Among those systems, few of those systems were selected for further reference related to the project domain. By looking at those similar systems, it is easy to get an idea of what technologies they have used in their systems, what functions they have used, the features and convenience they have provided to the user in those systems. To solve the problems mentioned in the first chapter. This chapter explains about those systems. These include domestic systems as well as foreign systems. They are Pharmacy2, UNION Chemist Pharmacy, Apollo Pharmacy and Health Guard Pharmacy.

2.2Similar systems

Pharmacy2

They use a web-based system with multiple functionalities to efficiently manage their pharmacy items, including drugs and health-related equipment. The system includes category management functions, making it easy for users to search for available items by viewing the categories provided. These categories include tablets, vitamins, supplements, medical devices, medical equipment, pet items, and more. Additionally, the search bar functionality enables users to find items quickly without unnecessary delay. Pharmacy2 offers various features such as ordering and online delivery, improving the overall user experience. The independent application of the E Medicare system significantly improves the efficiency of the tasks performed by employees compared to the previous paper-based system. It is developed with several functionalities that can be easily navigated by the employees. E Medicare system provides a separate login for specific users (employees), ensuring system security and protecting sensitive information such as staff details. This approach minimizes the chance of missing records, and the system facilitates faster bill generation with a built-in calculation function that ensures data accuracy. The E Medicare system allows users to quickly check the availability of drugs and medical equipment. Compared to the Pharmacy2 system, it offers a more user-friendly interface, makes data entry attractive and ensures user engagement. Notably, the E Medicare system uses strong security measures to protect against data theft. While the Pharmacy2 system provides item screening, it may lack systematic item classification, distinguishing the E Medicare system as exceptionally user-friendly. [1]

Union chemist pharmacy

Union chemist pharmacy has a web-based system to run their business effectively and efficiently. Union chemist pharmacy provides several functionalities in their web-based system such as categories, search products, add carts, etc. Under categories, there are several categories like Elderly Care, Baby Care and Blood Glucose. user. Items related to categories can be easily searched. Also, the online shopping functions provided by them are very useful for the user to buy the products easily. In addition to the search bar, help the customer find products without having to search here and there. As a stand-alone application, the E Medicare system provides more functionality that an inventory management system should have. Category, bill generation and report generation etc. When it comes to bill generation, the employee can use the system without stress as it is easy for the user to generate the bill without allocating separate time for calculation.

As it is a light version app compared to other systems, user can use the app without any delay compared to Union chemist pharmacy E Medicare fills several gaps like bill generation, report generation, attractive interface etc. Billing is done by the user without any stress, and the main advantage of the system in comparison with others is security. As mentioned, it ensures the security of the system by adopting encryption methods, the security of the system is important to ensure the security of some information stored in the databases like staff details. There are also similar functions such as checking category variants Available items, it is finally clear that both systems provide similar functions and there are other differences. [2]

apollo pharmacy

Apollo Pharmacy is a special pharmacy because it is an online pharmacy. All transactions and transactions are done by them through online modes. In that case, the entire business is maintained in a web base system. They provide many functions. Some of them are products, free shipping, forum questions and discounts. They use the question forum provided by them to communicate with the system users and by using the free shipping function, medicines and medical equipment can be ordered through online methods. And if there are discounts for some drugs, it provides a sub-function to show the discounts. In addition, the customer provides a registration function and can search for items by typing on the search bar.

As mentioned, the E Medicare system is implemented as a solution to the perceived problems with the traditional paper-based system. The system provides some functions within a scope to help users. Employees are the users who use the system. Comparing the E Medicare system with the Apollo Pharmacy system is more difficult, but when there are differences, the E Medicare system uses both systems to manage a pharmacy, but the E Medicare system can generate a report and calculate bills, and the developers of the E Medicare system thought about the security of the system. Some encryptions. Methods such as password checking have been adopted. So, there is a clear difference between the E Medicare system and the Apollo Pharmacy system. [3]

Health guard Pharmacy

Leveraging a web-based system, Health guard Pharmacy efficiently manages its business operations. The system encompasses various functionalities, including categories, search products, and an add-to-cart feature. Under categories, customers can easily navigate through sections such as adult care, baby care, and blood glucose, streamlining the search for related items. The online shopping function provided by Health guard Pharmacy proves beneficial for customers, offering a convenient way to purchase products. The search bar further enhances the customer experience by facilitating quick product discovery. In contrast, the E Medicare system, functioning as a standalone application, offers an extensive array of functionalities surpassing typical inventory management systems. Notable features include category management, bill generation, and report generation. Generating bills is simplified, allowing users to complete the process without separate calculations, reducing stress for employees. The E Medicare system, being a lightweight application, ensures smooth usage without lags. In comparison to Health guard Pharmacy, E Medicare excels in areas like bill and report generation, providing an attractive interface, and ensuring security through encryption methods. The emphasis on security is crucial for safeguarding sensitive details stored in databases, such as staff information. While both systems share similar functions, E Medicare stands out with its additional features and a robust security framework. [4]

Comparison Table

Requirements	System 01 Pharmacy2U	System 02 Healthguard Pharmacy	System 03 UNION CHEMIST PHARMECY	System 04 Apollo pharmacy	Manual System	Proposed System
Staff registration					no	yes
Category	yes	yes	yes	yes	no	yes
Manage products	yes	yes	yes	yes	no	yes
order	yes	yes	yes	yes	yes	yes
Inquiry	yes	yes	yes	yes	yes	yes
Search items	yes	yes	yes	yes	yes	yes

Summary

Compared to other systems, E Medicare provides different and similar functionalities to help the user, although the users of those systems are different, it works to manage a pharmacy effectively and efficiently. Also, in terms of similar functionality that those systems provide, search by search item categories or search column type and some systems provide both. In addition to many of those systems, provide the user with the function of checking existing drugs and health-related devices. E Medicare pharmacy checks provide a separate report to the user of dispensed items and inventory details. When it comes to the various E Medicare systems and other systems, the E Medicare system fills a gap even though it is a single application. The gaps filled by the system are as follows. Bill generation can be done by the user through the system which helps the user to calculate the bills easily without having to calculate the bills by writing paper. And the system provides a sales report of the shop to see the profit of the business, moreover, it manages the staff and because of the security encryption methods used in the system, only authorized people can enter the system, so the user can work the system with confidence. In addition, it helps to ensure the security of some important information such as staff details.

Chapter 03 Systems Analysis

3.1 Introduction

In this system analysis chapter, it clearly explains about what the solutions are according to the identified problems. There are three types of requirements. They are functional requirement, non-functional requirements, and system requirements. This chapter clearly explains those requirements within the scope of the project. System requirements, also known as software requirements, are a description of the target system's characteristics and functions. This category includes requirements that are linked to the functionality of software. This category includes requirements that are not linked to the software's functionality.

3.2 System requirements

In this part, clearly state the features and functional requirements of the target system. One of the E Medicare system's primary requirements is handling inventory management. There are many kinds of workers who do different jobs, including inventory management accountants, pharmacists, helpers, and system administrators. The roles are as follows.

1. Admin
2. Pharmacist
3. Assistant
4. Accountant

As mentioned earlier, employees can log into the system using their username and password. They can see the interface, but some employees are assigned a specific task by the system.

staff registration

The module specifically designed to manage the personnel registration procedure is called the staff registration module. The administrator should add the system and register them. Once registered, the employees will receive their username and password from the administrator, who can also change and remove employee details. Employees can access the system by entering their username and password, however because job functions vary, their ability to utilize the system is restricted. The following is the working scope

Admin Tasks

- Admin is responsible for staff registration and admin should give a username and password to the user after registration
- All the functions can be done through the system by admin.

Pharmacist Tasks

- Products are handled by pharmacist
- Categories are managed by pharmacist
- Check available items

Assistant Tasks

- Customer information is handled by the assistant

Accountant Task

- Order management is done by the accountant
- Generate invoices

Category Management

A category can be added, edited, or removed from the system using the category management module. They offer a wide range of categories for sale. These include vitamins, supplements, medications, pet supplies, medical equipment, and machines.

product management

There are many products in the product management module, which are classified into different categories. The supplement category includes antioxidants, multivitamins, fiber, selenium, glucosamine, and chondroitin; The vitamin group includes vitamins A, C, D, E, K, and B2 (riboflavin) and B3 (niacin). The medical devices category includes blood pressure monitors, blood pressure testers, scales, and other items. There are different brands of those things, and the price varies by brand. In addition, products can be added, edited, and removed from the system. They can also be searched by categories in the search box.

Customer Management

To place orders, this module records customer information. An assistance is provided to record the customer's phone number, address, and ID in this module.

Order Management

This order management module will generate the sales report and calculate the customer invoice. Additionally, supplies will be updated after a consumer purchases medication. Lastly, the user can view the customer bill in this module.

Functional Requirements

- Checking the username and password when entering the system
- Admin can add delete, edit employees, and view employee details
- Admin can generate staff detail report.
- Categories can be added, edit, delete through the system by and available categories can view in table
- add, edit, delete products through the system and available categories can view in a table
- check available items and items can sort category wise.
- place the order by recording customer details and assistant can view details after adding

Non-Functional Requirements

Security

As previously indicated, a working password and username are needed for a user to log into the system. The system cannot be logged into if the password or username is incorrect. The system will display a notice such as "You have entered invalid Username or Password, try again" if someone enters an incorrect username or password.

Accessibility

Passwords and usernames are generated based on each employee's employment position. While others cannot, the admin has access to every module and can use any feature. The product, category, and home management modules are accessible to the pharmacist. Furthermore, the pharmacist is allowed to work within that scope, but the assistant and accountant are limited to logging into the customer management and order management modules, respectively.

Flexibility

The system provides a colorful interface that anyone can attract to the system. And system provide a good experience to the user to work with the system without any stress by using picture and colorful buttons

User-friendliness

All the system's functions are easily understood in about a minute because to its user friendly language. Furthermore, the system performs a great deal of tasks automatically. All the accountants must do is click a button to view the most recent sale; the bill will compute automatically when the accountant needs to generate it. Additionally, the system automatically reduces the sold items whenever any of the things are sold.

Summary

Within the identified scope of the project, those are the functional requirement, non-functional requirement and system requirement in the system as mentioned above. Mainly there are five modules they are Staff management, Category management, Product management, and Order management. In the next chapter is explain about system design under the system design there are flow charts and pseudo code to explain the system.

Chapter 04 System Design

In this chapter, clearly describe the system design. Mainly use flowchart and mock code to explain system design. Flowchart and mock code clearly describe processes like staff management, batch management, product management, customer management, order management in E Medicare system. A flowchart is a diagram that shows how a workflow or process works. Pseudocode is a constructed, informal language that assists programmers in developing algorithms. The mock code and flowchart help to understand the scenario in the system and how they work as separate modules. This situation is easy to understand even without any technical knowledge. So even non-technical people can understand the situation.

In the logging process, every employee has separate logging and every employee category have separate view in the system. After completing the task, the employee can logout from the system.

4.1 Staff Registration

In the staff management process, employees can be added, edited, and deleted when a certain user logs into the system. The user must first supply the system with their username, password, complete name, user ID, and role classification. The system will display the message "Successfully added" if the user's inputs are legitimate, at which point the user can add an employee. If not, an alert labeled "Invalid data" will appear on the screen. Additionally, the system needs to modify the input box if the user must change an employee's details, and then save the changes. If the user needs to delete an employee, the system allows for that as well.

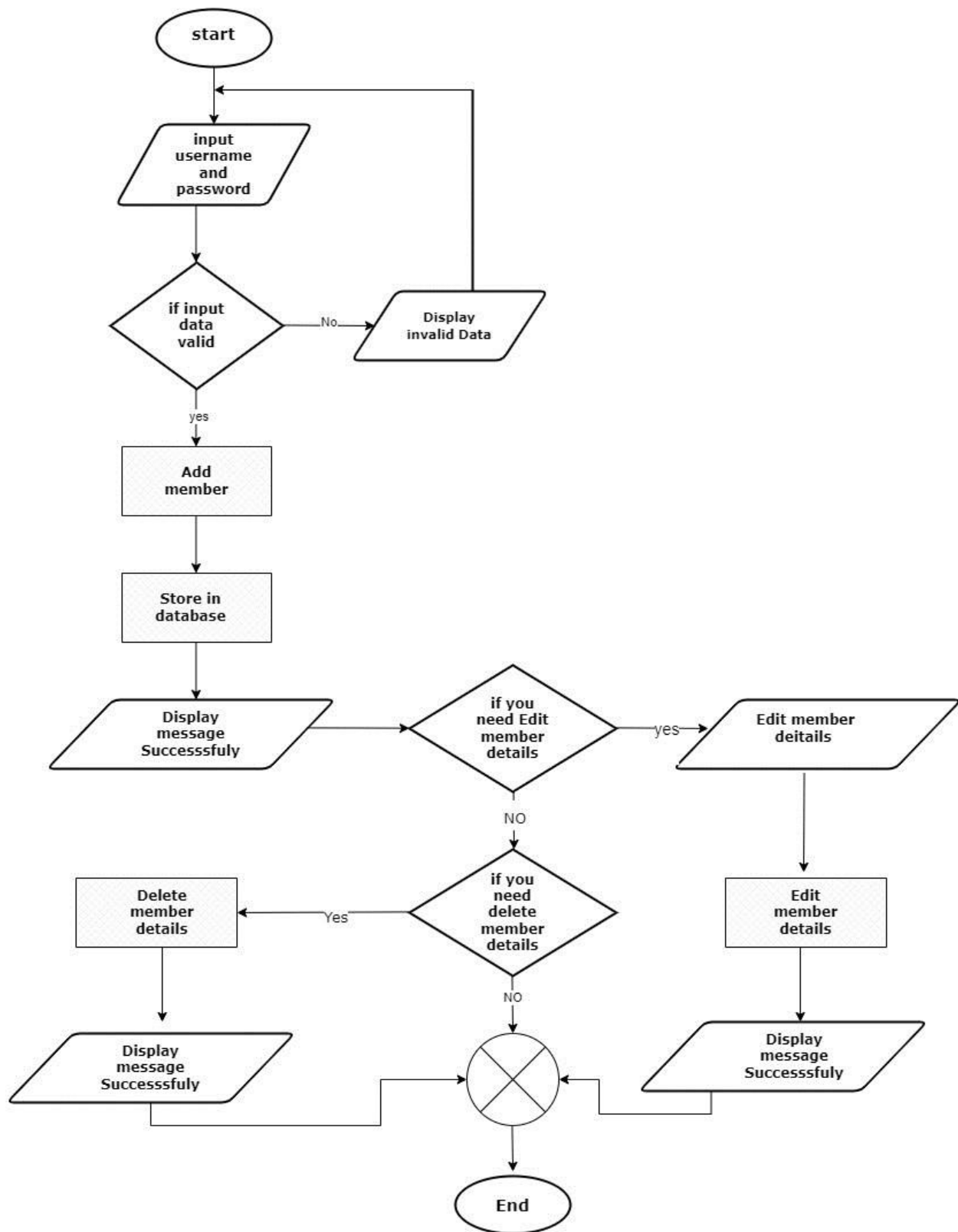


Figure 1Staff Registration

Category Management

In the category management process, when user logging to the system user can add, edit, and delete categories. First, users of the system need to provide category name and category code to the system. If inputs that provide by the users are valid user can add category, once the category adding process is done, system popup message “Successfully added” and if invalid, the system will popup alert as “invalid data”. Furthermore, if user need to edit categories details system requires to edit input box and save. if successfully edited it shows successfully edited and if user need delete category, it possible in the System.

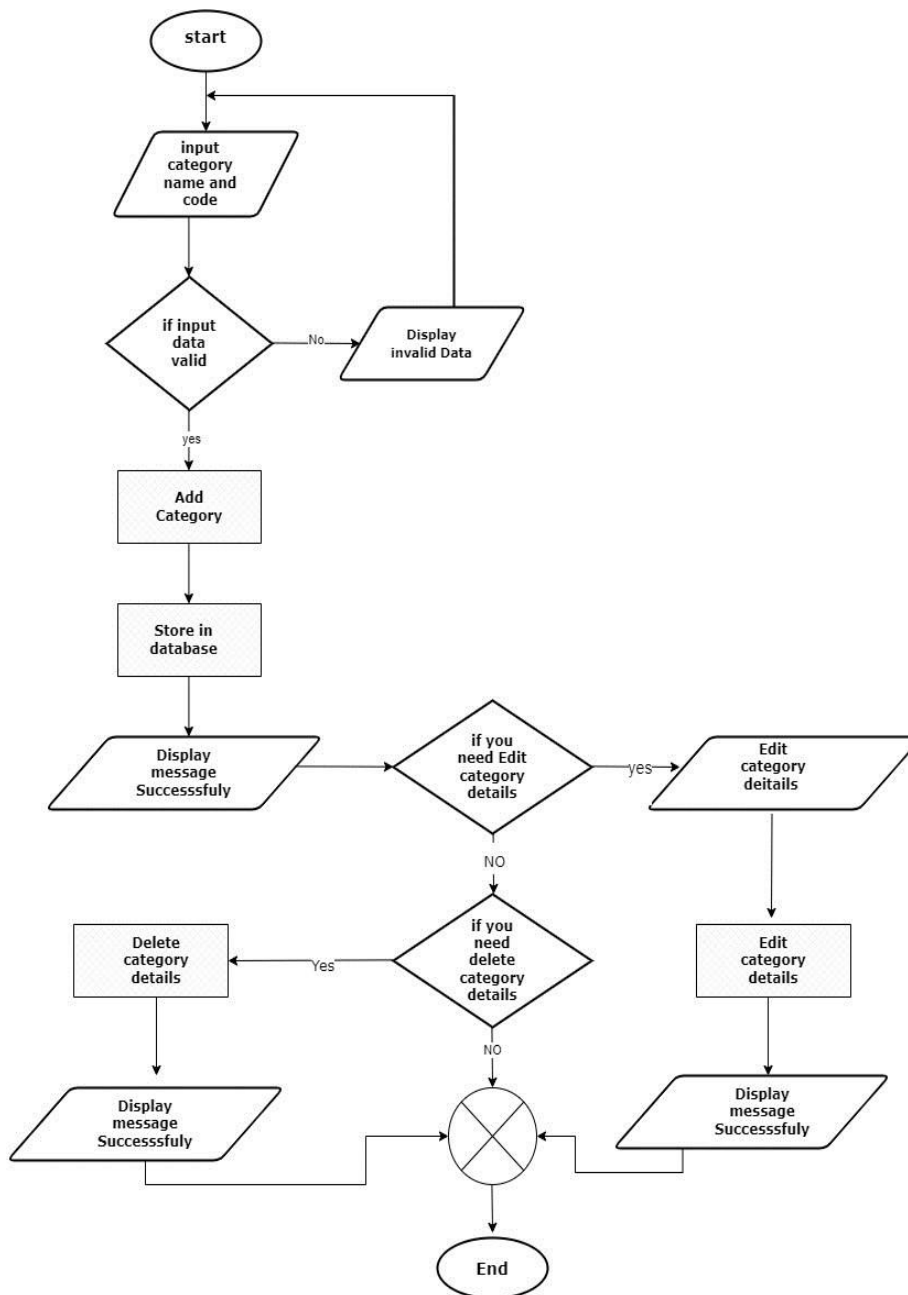


Figure 2Category Management

Product Management

Products can be added, edited, and deleted during the product management process by logging into the product management form. The product code, name, price, quantity, category, and description must first be entered into the system by the user. When users submit valid inputs, the system notifies them that the adding process has been completed and displays the message "Successfully updated." If the inputs are invalid, the system notifies the user that the data is invalid. Additionally, the system needs to modify the input box and store any changes made by the user for the product data to be updated. If the editing is successful, this is indicated, and the user can also delete the product from the system.

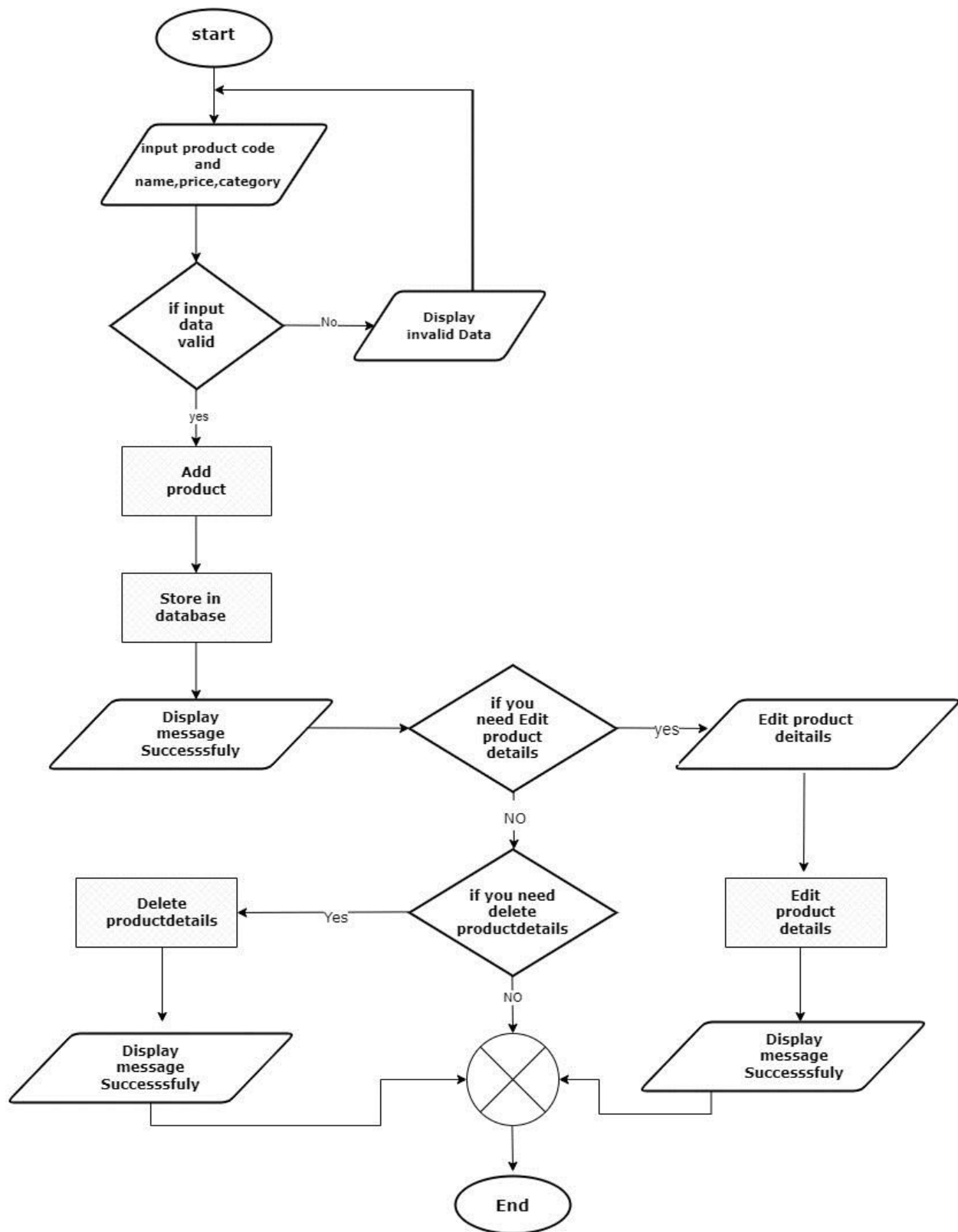


Figure 3Product Management

Customer Management

In the customer management process, also provide same functionality to user, if the user enters to the customer management form. It is possible to add, edit and delete customers. First, users of the system need to provide customer phone number, customer id and customer address to the system. If inputs that provide by the users are valid user can add customer, then the adding process is complete and system popup message “Successfully updated” and if invalid, the system will popup alert as “invalid data”. Furthermore, if user need to edit customer details, system requires editing the input boxes. if successfully edited it shows successfully edited. if user need to delete customer it also possible in the system.

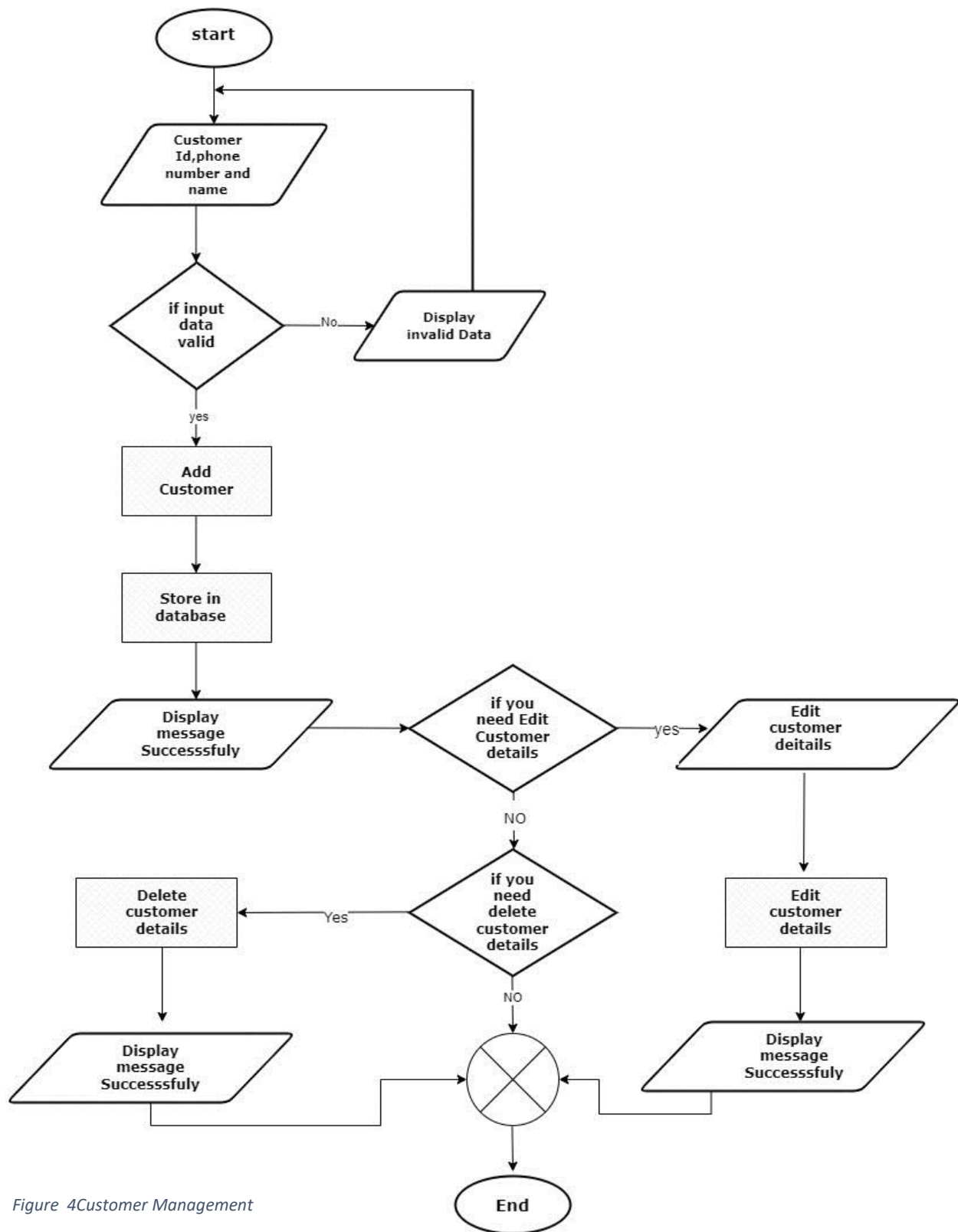


Figure 4Customer Management

order management

In order management process when the user enter the order management form, the user can do calculate bill and report generate. Firstly, user need to input order number, quantity, order date. then if data valid system retrieves product name, customer id, customer address from database. and the process the order and calculate the bill. And if order success fully processed the database update automatically. And system print the bill. If user need view the report, it also possible.

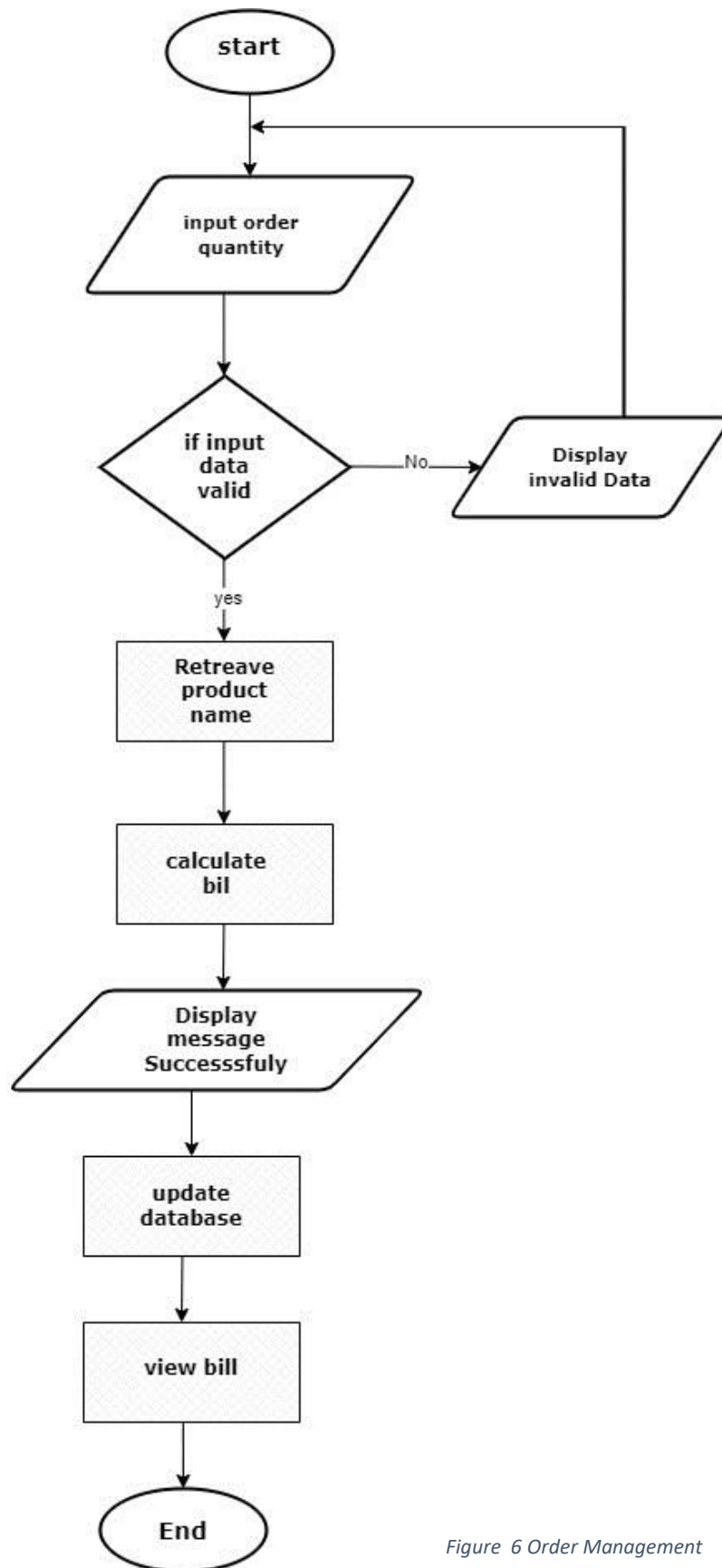


Figure 6 Order Management

Pseudocode

Logging

Begin

Input username. password

While username and password is incorrect do

Display Incorrect username or password

Input username. Password

end while

If user category = "admin" then

admin view

else if category = "pharmacist" then

pharmacist view

else if category = "assistant" then

assistant view

else if category ="accountant" then

accountant view

end if

Logout

End

Staff Management

Begin

Input username, full name, staff categories, password, telephone

While user enter incorrect data do

Display invalid data

Input username, full name, staff categories, password, telephone

Staff details added to database

Display successfully updated
If user need edit staff details, then

 Edit staff details

 Display staff member successfully edited

 Update database

else If user need delete staff details then

 Delete staff details

 Display staff member successfully deleted

 Update database

End if

End

Category Management

Begin

 Input Category name, Category code

 While user enter incorrect data do

 Display invalid data

 Input category name, category code

 Category details added to database

 Display successfully updated

 If user need edit Category details, then

 Edit Category details

```
        Display successfully
        Update database
    else If user need delete Category details, then
        Delete Category details
        Display successfully deleted Category details
        Update database
    End if
End
```

Product Management

```
Begin
    Input product name, product price, quantity, description
    While user enter incorrect data do
        Display invalid data
    Input product name, product price, quantity, description
    Product details added to database
    If user need edit product details, then
        Edit product details
        Display product successfully edited
    Else if need delete product details, then
        Delete product details
        Display product successfully delete
        Update database
    End if
End
```

Customer Management

Begin

Input customer id, customer phone number

While user enter incorrect data do

Display invalid data

Input customer id, customer phone number

Customer details added to database

Display successfully update

If user need edit details, then

Edit details

Display customer successfully edited

Update database

Else if user need delete details

Delete customer details

Display successfully delete

Update database

End if

End

Order Management

Begin

Input order number, quantity, order date

While input data valid

Display invalid data

Input order number, quantity, order date

```
End while
Retrieve product name, customer id in database
    Process order and calculate bill
        Display order successfully added
            Update database
            Print bill
            if user need view report, then
                Display report
            End if
        End if
    End
```

Appendix

Login interfaces



Figure 4 login interface

Code segments

Login Form Code Segment

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using MySql.Data.MySqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace E_Medicare_Management
{
    public partial class Login : Form
    {
        public static string userRole;
        public Login()
        {
            InitializeComponent();

            MySqlConnection Con = new
            MySqlConnection("server=localhost;database=medicare;username=root;password=Azone12
            3");

            private void btnLogin_Click(object sender, EventArgs e)
            {
                Con.Open();
                MySqlDataAdapter sda = new MySqlDataAdapter("Select Count(*) from
                Staff where UserId = '" + txtUserID.Text + "' and Password = '" + txtPassword.Text
                + "' and Role = '" + cmbRole.Text + "'", Con);
                DataTable dt = new DataTable();
                sda.Fill(dt);
                if (dt.Rows[0][0].ToString() == "1")
                {
                    if (cmbRole.SelectedItem.ToString() == "Admin" ||
                    cmbRole.SelectedItem.ToString() == "Assistant" || cmbRole.SelectedItem.ToString() ==
                    "Accountant" || cmbRole.SelectedItem.ToString() == "Pharmacist")
                    {
                        E_MEd home = new E_MEd(cmbRole.SelectedItem.ToString());
                        home.Show();
                        this.Hide();
                    }
                }
                else
                {
                    MessageBox.Show("can't log");
                }
            }
        }
    }
}
```

```

        else
        {
            MessageBox.Show("Wrong UserName, Password or User Role");
        }
        Con.Close();
    }

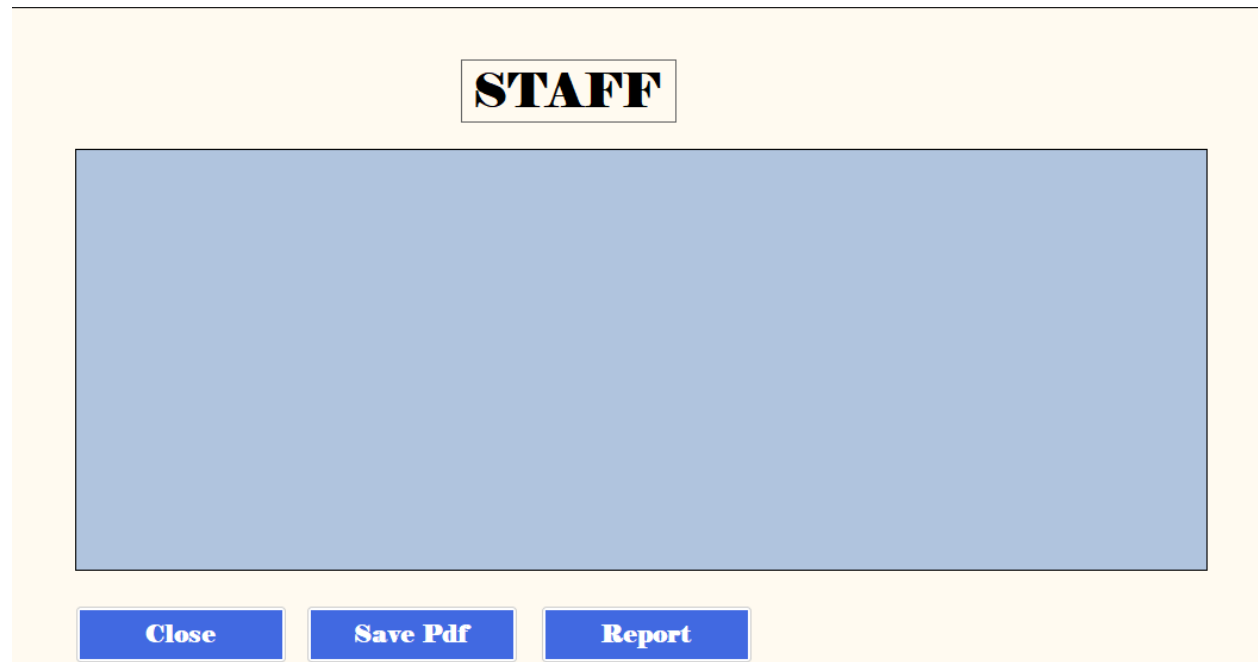
    private void checkBoxshow_CheckedChanged(object sender, EventArgs e)
    {
        if (checkBoxshow.Checked)
        {
            txtPassword.UseSystemPasswordChar = true;
            var checkBox = (CheckBox)sender;
            checkBoxshow.Text = "Show";
        }
        else
        {
            txtPassword.UseSystemPasswordChar = false;
            var checkBox = (CheckBox)sender;
            checkBoxshow.Text = "Hide";
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        this.Close();
        System.Windows.Forms.Application.Exit();
    }

    private void Login_Load(object sender, EventArgs e)
    {
        txtPassword.UseSystemPasswordChar = true;
    }
}

```

Staff Report



The screenshot shows a web interface for a staff report. At the top, there is a yellow header bar. Inside this bar, on the right side, is a black rectangular box with the word "STAFF" written in bold, black, uppercase letters. Below the header bar is a large, empty blue rectangular area, which is likely a placeholder for the report data. At the bottom of the interface, there are three blue buttons with white text: "Close", "Save Pdf", and "Report".

Figure 5 staff report interface

Staff Report Form Code Segment

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using MySql.Data.MySqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace E_Medicare_Management
{
    public partial class StaffReport : Form
    {
        public StaffReport()
        {
            InitializeComponent();
        }
        MySqlConnection Con = new
        MySqlConnection("server=localhost;database=medicare;username=root;password=Azone123");

        void populateoders()
        {
            try
            {
```

```

        Con.Open();
        string Myquery = "select * from Staff";
        MySqlDataAdapter da = new MySqlDataAdapter(Myquery, Con);
        MySqlCommandBuilder builder = new MySqlCommandBuilder(da);
        var ds = new DataSet();
        da.Fill(ds);
        dgvStaffReport.DataSource = ds.Tables[0];
        Con.Close();
    }
    catch
    {
    }
}

private void dgvStaffReport_CellClick(object sender, DataGridViewCellEventArgs e)
{
    printPreviewDialog1.Document = printDocument1;
    if (printPreviewDialog1.ShowDialog() == DialogResult.OK)
    {
        printDocument1.Print();
    }
}

private void btnview_Click(object sender, EventArgs e)
{
    this.Hide();
}

private void StaffReport_Load(object sender, EventArgs e)
{
    populateorders();
}

private void printDocument1_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
{
    e.Graphics.DrawString("E Medi STAFF MEMBER", new Font("Century", 25,
FontStyle.Bold), Brushes.Red, new Point(160, 10));
    e.Graphics.DrawString(DateTime.Now.ToString(), new Font("Century", 16,
FontStyle.Regular), Brushes.Black, new Point(200, 50));

    e.Graphics.DrawString("_____
_____
", new Font("Century", 25, FontStyle.Bold),
Brushes.Black, new Point(-120, 80));
    e.Graphics.DrawString("STAFF MEMBER ID-----: " +
dgvStaffReport.CurrentRow.Cells[0].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 180));
    e.Graphics.DrawString("STAFF MEMBER NAME-----: " +
dgvStaffReport.CurrentRow.Cells[1].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 210));
    e.Graphics.DrawString("STAFF ROLE-----: " +
dgvStaffReport.CurrentRow.Cells[2].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 240));
}

```

```

        e.Graphics.DrawString("CONTACT -----: " +
dgvStaffReport.CurrentRow.Cells[3].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 270));
        e.Graphics.DrawString("PASSWORD-----: " +
dgvStaffReport.CurrentRow.Cells[4].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 300));

e.Graphics.DrawString("_____
_____
", new Font("Century", 25, FontStyle.Bold),
Brushes.Black, new Point(-120, 400));
        e.Graphics.DrawString("E MEDICARE ", new Font("Century", 25, FontStyle.Bold),
Brushes.Green, new Point(230, 480));
        e.Graphics.DrawString("+", new Font("Forte", 85, FontStyle.Bold),
Brushes.Red, new Point(400, 490));
    }

private void btnpdf_Click(object sender, EventArgs e)
{
    PrintDialog printDialog1 = new PrintDialog();
    printDialog1.Document = printDocument1;

    DialogResult result = printDialog1.ShowDialog();

    if (result == DialogResult.OK)
    {
        printDocument1.Print();
    }
}

private void button1_Click(object sender, EventArgs e)
{
    printPreviewDialog2.Document = printDocument2;
    int height = dgvStaffReport.Height;
    dgvStaffReport.Height = dgvStaffReport.RowCount *
dgvStaffReport.RowTemplate.Height * 2;
    bmp = new Bitmap(dgvStaffReport.Width, dgvStaffReport.Height).
dgvStaffReport.DrawToBitmap(bmp, new Rectangle(30, 10, dgvStaffReport.Width,
dgvStaffReport.Height));
    dgvStaffReport.Height = height;
    printPreviewDialog2.ShowDialog();
}

Bitmap bmp;
private void printDocument2_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
{
    e.Graphics.DrawImage(bmp, 0, 0);
}
}
}

```

Product Report



Figure 6 product Report interface

Product Report Form Code Segment

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using MySql.Data.MySqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace E_Medicare_Management
{
    public partial class Productreport : Form
    {
        public Productreport()
        {
            InitializeComponent();
        }
        MySqlConnection Con = new
        MySqlConnection("server=localhost;database=medicare;username=root;password=Azone123");

        void populateoders()
```

```

{
    try
    {
        Con.Open();
        string Myquery = "select * from Product";
        MySqlDataAdapter da = new MySqlDataAdapter(Myquery, Con);
        MySqlCommandBuilder builder = new MySqlCommandBuilder(da);
        var ds = new DataSet();
        da.Fill(ds);
        dgvViewProduct.DataSource = ds.Tables[0];
        Con.Close();
    }
    catch
    {
    }
}

private void Productreport_Load(object sender, EventArgs e)
{
    populateorders();
}

private void dgvViewProduct_CellClick(object sender, DataGridViewCellEventArgs e)
{
    printPreviewDialog1.Document = printDocument1;
    if (printPreviewDialog1.ShowDialog() == DialogResult.OK)
    {
        printDocument1.Print();
    }
}

private void btnClose_Click (object sender, EventArgs e)
{
    this.Hide();
}

private void printDocument1_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
{
    e.Graphics.DrawString("E MEDICARE PRODUCT DETAILS", new Font("Century", 25,
FontStyle.Bold), Brushes.Green, new Point(160, 10));
    e.Graphics.DrawString(DateTime.Now.ToString(), new Font("Century", 16,
FontStyle.Regular), Brushes.Black, new Point(200, 50));

    e.Graphics.DrawString("_____
_____
", new Font("Century", 25, FontStyle.Bold),
Brushes.Black, new Point(-120, 80));
    e.Graphics.DrawString("PRODUCT CODE-----: " +
dgvViewProduct.CurrentRow.Cells[0].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 180));
    e.Graphics.DrawString("PRODUCT NAME-----: " +
dgvViewProduct.CurrentRow.Cells[1].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 210));

```

```

        e.Graphics.DrawString("PRODUCT PRICE-----: " + "Rs " +
dgvViewProduct.CurrentRow.Cells[2].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 240));
        e.Graphics.DrawString("QUANTITY-----: " +
dgvViewProduct.CurrentRow.Cells[3].Value.ToString() , new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 270));
        e.Graphics.DrawString("PRODUCT DESCRIPTION-----:" +
dgvViewProduct.CurrentRow.Cells[4].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 300));
        e.Graphics.DrawString("CATEGORY CODE-----: " +
dgvViewProduct.CurrentRow.Cells[5].Value.ToString(), new Font("Century", 20,
FontStyle.Regular), Brushes.Black, new Point(80, 330));

e.Graphics.DrawString("_____
_____
_____
", new Font("Century", 25, FontStyle.Bold),
Brushes.Black, new Point(-120, 400));
        e.Graphics.DrawString("E MEDICARE ", new Font("Century", 25, FontStyle.Bold),
Brushes.Green, new Point(230, 480));
        e.Graphics.DrawString("+", new Font("Forte", 85, FontStyle.Bold),
Brushes.Red, new Point(400, 490));
    }

    private void btnprint_Click(object sender, EventArgs e)
    {
        PrintDialog printDialog1 = new PrintDialog();
        printDialog1.Document = printDocument1;

        DialogResult result = printDialog1.ShowDialog();

        if (result == DialogResult.OK)
        {
            printDocument1.Print();
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        printPreviewDialog2.Document = printDocument2;
        int height = dgvViewProduct.Height;
        dgvViewProduct.Height = dgvViewProduct.RowCount *
dgvViewProduct.RowTemplate.Height * 2;
        bmp = new Bitmap(dgvViewProduct.Width, dgvViewProduct.Height);
        dgvViewProduct.DrawToBitmap(bmp, new Rectangle(30, 10, dgvViewProduct.Width,
dgvViewProduct.Height));
        dgvViewProduct.Height = height;
        printPreviewDialog2.ShowDialog();
    }

    Bitmap bmp;
    private void printDocument2_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
    {
        e.Graphics.DrawImage(bmp, 0, 0);
    }
}

```

Loadig Form

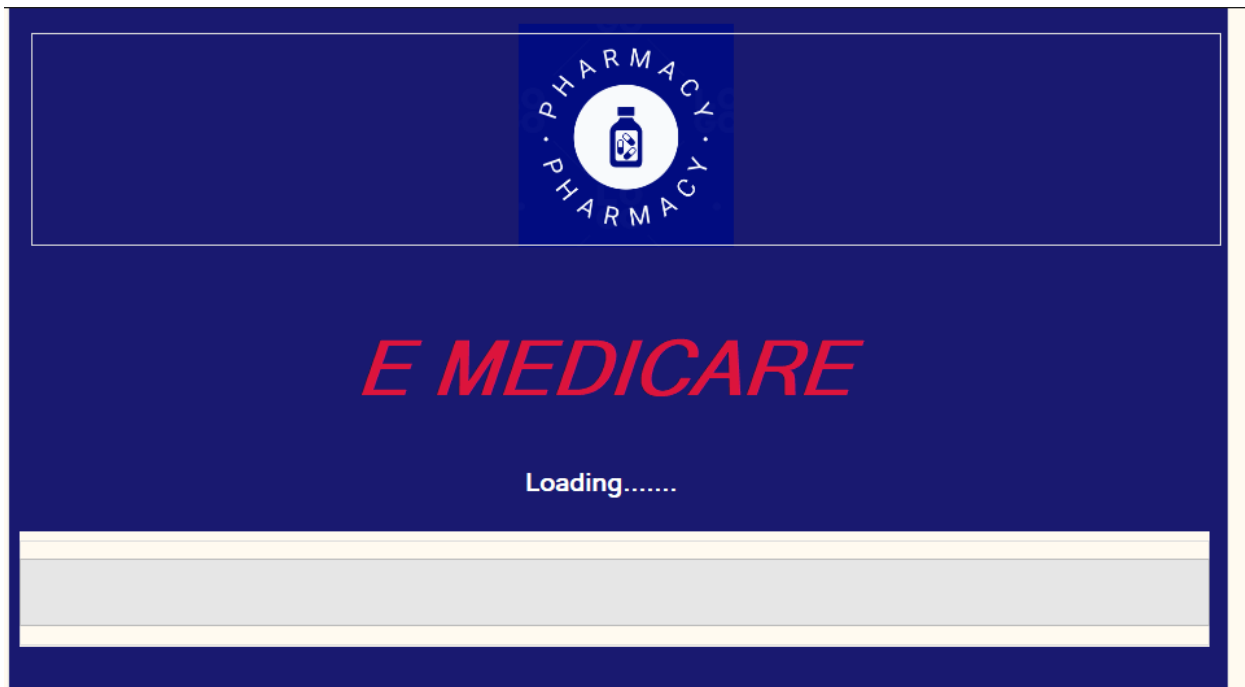


Figure 7 Loding interface

loading Form Code Segment

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace E_Medicare_Management
{
    public partial class Load : Form
    {

        public Load()
        {
            InitializeComponent();
            timer1.Start();
        }
        int startPoint = 0;

        private void timer1_Tick(object sender, EventArgs e)
        {
```

```

startPoint += 1;
progressBarpro.Value = startPoint;
if (progressBarpro.Value == 100)
{
    progressBarpro.Value = 0;
    timer1.Stop();
    Login login = new Login();
    this.Hide();
    login.ShowDialog();
}
}
}
}

```

Form Code Segment

- Staff registration, Category Management, Product Management, Customer Management, Order Management are all coded using one form in Visual Studio.

ORDER MANAGEMENT

CATEGORY ▾

ProductCode	Product Name	Product Price
122	vitamin	2000
*		

Quantity

CUSTOMER

CustomerName	CustomerContact	CustomerAddress
d.kamal	75345673	kurunegala
D.alvia	762345673	kandy
*		

Order ID

Customer Name

Product Code

Product Name

Unit Price

Order Date

Insert Oder View Oders Monday , December ▾

Num	Product	Quantity	UnitPrice
*			

TOTAL RS 0.00

ADD ORDER

Figure 8 order interface

PRODUCT MANAGEMENT

Product Code

Product Name

Product Price

Quantity

Product Description

Category

SEARCH REFRESH

ProductCode	Product Name	Product Price	Quantity	Product
122	vitamin	2000	0	vitamin
*				

Add Edit Delete

Product Report

Figure 9 product management interface

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Collections;

namespace E_Medicare_Management
{
    public partial class E_MEd : Form
    {
        dba db;
        public static DataTable st = new DataTable();
        public static DataTable ct = new DataTable();
        public static DataTable catt = new DataTable();
        public static DataTable pt = new DataTable();
        public static DataTable tab = new DataTable();

        public static string CategoryName;
        public static string Role;

        public static ArrayList Cat = new ArrayList();
        public static ArrayList stff = new ArrayList();

        int flag;
        int stock;
        int uprice, totprice, qty;
        string product;
        int num = 0;
        string cname;

        private int count = 0;

        void updateproduct()
        {
            int id = Convert.ToInt32(dgvorder.CurrentRow.Cells[0].Value.ToString());
            int newQty = stock - Convert.ToInt32(txtorderedquantity.Text);
            if (newQty < 0)
            {
                MessageBox.Show("Operation Failed");
            }
            else
            {

```

```

        db.cud("update Product set Quantity = " + newQty + " where ProductCode = " + id + ";");
        pt.Clear();
        db.ProdctTable("select * from Product");
        dgvcatogory.DataSource = pt;
    }
}
void staffrefresh()
{
    St. Clear();
    db.staffTable("select*from Staff");
    dgvstaff.DataSource = st;
}
void productrefresh()
{
    pt.Clear();
    db.ProdctTable("select*from Product");
    dgvproduct.DataSource = pt;
}

private void lblStaff_Click(object sender, EventArgs e)
{
    panelStaff.Visible = true;
    panelcustomer.Visible = false;
    panelCategory.Visible = false;
    panelProduct.Visible = false;
    panelOrder.Visible = false;

    cmbStaffsearch.Items.Clear();
    db.retrieve("Select Role from Staff", "Staff");
    foreach (string e in stff)
    {
        cmbStaffsearch.Items.Add(rv);
    }
    stff.Clear();
}

public E_MEd(string userRole)
{
    InitializeComponent();
    db = new dba();

    db.staffTable("select * from Staff");
    db.customerTable("select * from Customer");
    dgvstaff.DataSource = st;
    dgvCus.DataSource = ct;
    db.CategoryTable("select * from Category");
    dgvCategory.DataSource = catt;
    db.ProdctTable("select * from Product");
    dgvproduct.DataSource = pt;
    dgvcatogory.DataSource = pt;
    dgvcusto.DataSource = ct;
}

```

```

        db = new dba();
        panelStaff.Visible = false;
        panelcustomer.Visible = false;
        panelCategory.Visible = false;
        panelProduct.Visible = false;
        panelOrder.Visible = false;

        if (userRole == "Pharmacist")
        {
            lblStaff.Visible = false;
            lblCustomerAd.Visible = false;
            lblOrderAd.Visible = false;
        }
        else if (userRole == "Accountant")
        {
            lblStaff.Visible = false;
            lblCustomerAd.Visible = false;
            lblProductAd.Visible = false;
            lblCategoryAd.Visible = false;
        }
        else if (userRole == "Assistant")
        {
            lblStaff.Visible = false;
            lblProductAd.Visible = false;
            lblCategoryAd.Visible = false;
            lblOrderAd.Visible = false;
        }
    }

    private void lblCategoryAd_Click(object sender, EventArgs e)
    {
        panelCategory.BringToFront();
        panelCategory.Visible = true;
        panelStaff.Visible = false;
        panelcustomer.Visible = false;
        panelProduct.Visible = false;
        panelOrder.Visible = false;
    }

    private void lblCustomerAd_Click(object sender, EventArgs e)
    {
        panelcustomer.BringToFront();
        panelcustomer.Visible = true;
        panelStaff.Visible = false;
        panelCategory.Visible = false;
        panelOrder.Visible = false;
        panelProduct.Visible = false;
    }

    private void btnCatAdd_Click_1(object sender, EventArgs e)
    {
        db.cud("Insert into Category(CategoryCode, CategoryName) values('" +
            txtCategoryCode.Text + "','" + txtCategoryName.Text + "')");
    }

```

```

        catt.Clear();
        db.CategoryTable("select * from Category");
        dgvCategory.DataSource = catt;
    }

    private void lblCatDelete_Click(object sender, EventArgs e)
    {
        DialogResult dialog = MessageBox.Show("Do you need to delete", "Exit",
        MessageBoxButtons.YesNo);
        if (dialog == DialogResult.Yes)
        {
            db.cud("delete from Category where CategoryCode = '" +
            txtCategoryCode.Text + "'");
            catt.Clear();
            db.CategoryTable("select * from Category");
            dgvCategory.DataSource = catt;
        }
    }

    private void dgvCategory_CellContentClick(object sender,
    DataGridViewCellEventArgs e)
    {
        var demo = dgvCategory.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
        string s = demo.ToString();
        txtCategoryCode.Text =
        dgvCategory.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtCategoryName.Text =
        dgvCategory.Rows[e.RowIndex].Cells[1].Value.ToString();
    }

    private void btnProductAdd_Click(object sender, EventArgs e)
    {
        db.cud("Insert into Product(ProductCode, ProductName, ProductPrice, Quantity,
        ProductDescription, CategoryCode) values(" + txtProuctCode.Text + "," +
        txtProductName.Text + "," + txtProductPrice.Text + "," + txtQuantity.Text + "," +
        txtProDec.Text + "," + cmbCat.GetItemText(cmbCat.SelectedItem) + ")");
        pt.Clear();
        db.ProdctTable("select * from Product");
        dgvproduct.DataSource = pt;
    }

    private void lblProductAd_Click(object sender, EventArgs e)
    {
        panelProduct.BringToFront();
        panelProduct.Visible = true;
        panelStaff.Visible = false;
        panelcustomer.Visible = false;
        panelCategory.Visible = false;
        panelOrder.Visible = false;

        cmbCat.Items.Clear();
        cmbSearch.Items.Clear();
        db.retrive("Select CategoryName from Category", "Category");
        foreach(string e in Cat)
        {

```

```

        cmbCat.Items.Add(rv);
        cmbSearch.Items.Add(rv);
    }
    Cat.Clear();
}

private void lblSerch_Click(object sender, EventArgs e)
{
    pt.Clear();
    db.ProdctTable("Select*from Product where CategoryCode='" + cmbSearch.Text +
    """);
    dgvproduct.DataSource = pt;
}

private void btnProductEdit_Click(object sender, EventArgs e)
{
    DialogResult dialog = MessageBox.Show("Do you need to Edit", "Exit",
    MessageBoxButtons.YesNo);
    if (dialog == DialogResult.Yes)
    {
        db.cud("update Product set    ProductName='" + txtProductName.Text +
        "',ProductPrice=" + txtProductPrice.Text + ",Quantity=" + txtQuantity.Text +
        ",ProductDescription='" + txtProDec.Text + "',CategoryCode='" +
        cmbCat.GetItemText(cmbCat.SelectedItem) + "where ProductCode=" + txtProuctCode.Text);
        pt.Clear();
        db.ProdctTable("select * from Product");
        dgvproduct.DataSource = pt;
    }
}

private void dgvproduct_CellContentClick(object sender, DataGridViewCellEventArgs
e)
{
    var prod = dgvproduct.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
    string s = prod.ToString();
    txtProuctCode.Text = dgvproduct.Rows[e.RowIndex].Cells[0].Value.ToString();
    txtProductName.Text = dgvproduct.Rows[e.RowIndex].Cells[1].Value.ToString();
    txtProductPrice.Text = dgvproduct.Rows[e.RowIndex].Cells[2].Value.ToString();
    txtQuantity.Text = dgvproduct.Rows[e.RowIndex].Cells[3].Value.ToString();
    txtProDec.Text = dgvproduct.Rows[e.RowIndex].Cells[4].Value.ToString();
    txtCategoryCode.Text = dgvproduct.Rows[e.RowIndex].Cells[5].Value.ToString();
}

private void lblRefresh_Click(object sender, EventArgs e)
{
    productrefresh();
}

private void lblLogAd_Click(object sender, EventArgs e)
{
    this.Close();
    Login log = new Login();
    log.Show();
}

```

```

    }

    private void lblCusAdd_Click_1(object sender, EventArgs e)
    {
        db.cud("Insert into Customer(CustomerName, CustomerContact, CustomerAddress)
values('" + txtCustomerName.Text + "'," + txtCustomerPhone.Text + "," + txtCustomerAddress.Text + "')");
        ct.Clear();
        db.customerTable("select * from Customer");
        dgvCus.DataSource = ct;
    }

    private void dgvCus_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        var demo = dgvCus.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
        string s = demo.ToString();
        txtCustomerName.Text = dgvCus.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtCustomerPhone.Text = dgvCus.Rows[e.RowIndex].Cells[1].Value.ToString();
        txtCustomerAddress.Text = dgvCus.Rows[e.RowIndex].Cells[2].Value.ToString();
    }

    private void lblCusEdit_Click_1(object sender, EventArgs e)
    {
        DialogResult dialog = MessageBox.Show("Do you need to Edit", "Exit",
        MessageBoxButtons.YesNo);
        if (dialog == DialogResult.Yes)
        {
            db.cud("update Customer set CustomerContact=" + txtCustomerPhone.Text +
            ", CustomerAddress='" + txtCustomerAddress.Text + "' where CustomerName='" +
            txtCustomerName.Text + "'");
            ct.Clear();
            db.customerTable("select * from Customer");
            dgvCus.DataSource = ct;
        }
    }

    private void lblCusDel_Click_1(object sender, EventArgs e)
    {
        DialogResult dialog = MessageBox.Show("Do you need to delete", "Exit",
        MessageBoxButtons.YesNo);
        if (dialog == DialogResult.Yes)
        {
            db.cud("delete from Customer where CustomerName = '" +
            txtCustomerName.Text + "'");
            ct.Clear();
            db.customerTable("select * from Customer");
            dgvCus.DataSource = ct;
        }
    }

    private void lblOrderAd_Click(object sender, EventArgs e)
    {
        panelOrder.BringToFront();
        panelOrder.Visible = true;
        panelStaff.Visible = false;
        panelcustomer.Visible = false;
        panelCategory.Visible = false;
        panelProduct.Visible = false;
    }

```

```

cmboderedCat.Items.Clear();
db.retrive("Select CategoryName from Category", "Category");
foreach (string rv in Cat)
{
    cmboderedCat.Items.Add(rv);
}
Cat.Clear();
if (count == 0)
{
    count++;
    tab.Columns.Add("Num", typeof(int));
    tab.Columns.Add("Product", typeof(string));
    tab.Columns.Add("Quantity", typeof(int));
    tab.Columns.Add("UnitPrice", typeof(int));
    tab.Columns.Add("TotPrice", typeof(int));

    dgvorder.DataSource = tab;
}
}

private void dgvcategory_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    product = dgvcategory.CurrentRow.Cells[0].Value.ToString();

    stock = Convert.ToInt32(dgvcategory.CurrentRow.Cells[3].Value.ToString());
    uprice = Convert.ToInt32(dgvcategory.CurrentRow.Cells[2].Value.ToString());

    txtoderedpId.Text = dgvcategory.Rows[e.RowIndex].Cells[0].Value.ToString();
    txtProname.Text = dgvcategory.Rows[e.RowIndex].Cells[1].Value.ToString();
    txtUnitPr.Text = dgvcategory.Rows[e.RowIndex].Cells[2].Value.ToString();
    flag = 1;
}

private void dgvcusto_CellClick(object sender, DataGridViewCellEventArgs e)
{
    var demo = dgvcusto.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
    string s = demo.ToString();
    txtOrderedCusto.Text = dgvcusto.Rows[e.RowIndex].Cells[0].Value.ToString();
}

private void btnProductDelete_Click(object sender, EventArgs e)
{
    DialogResult dialog = MessageBox.Show("Do you need to delete", "Exit",
    MessageBoxButtons.YesNo);
    if (dialog == DialogResult.Yes) {
        db.cud("delete from Product where ProductCode = " +
        txtProuctCode.Text + "");
        pt.Clear();
        db.ProdctTable("select * from Product");
        dgvproduct.DataSource = pt;
    }
}

```

```

    }
    }

    private void btnInsertorder_Click(object sender, EventArgs e)
    {
        if (txtOrderId.Text == "" || txtOrderedCusto.Text == "" || txtoderedpId.Text
        == "" || lbltotalAmount.Text == "")
        {
            MessageBox.Show("Fill The Data Correctly");
        }
        else
        {
            try
            {
                db.cud("Insert into Orders (OrderId, Quantity,
                Orderdate,CustmerName,Total,ProductName,UnitPrice) values(" + txtOrderId.Text + "," +
                txtorderedquantity.Text + "," + orderdate.Text +
                "','" + txtOrderedCusto.Text + "','" + lbltotalAmount.Text + "','" + txtProname.Text + "','" + txtUnitPr.Te
                xt + ")");
            }
            catch
            {
                MessageBox.Show("Order Not Added");
            }
        }
    }

    private void label125_Click(object sender, EventArgs e)
    {
        int sum = 0;
        lbltotalAmount.Text = "0.0";
        for (int i = 0; i < dgvorder.Rows.Count; ++i)
        {
            sum += Convert.ToInt32(dgvorder.Rows[i].Cells[4].Value);
        }
        lbltotalAmount.Text = sum.ToString();
    }

    private void btnViewOrder_Click(object sender, EventArgs e)
    {
        ViewOrderReport view = new ViewOrderReport();
        view.Show();
    }

    private void btnCategorEdit_Click(object sender, EventArgs e)
    {
        DialogResult dialog = MessageBox.Show("Do you need to Edit", "Exit",
        MessageBoxButtons.YesNo);
        if (dialog == DialogResult.Yes)
        {
            db.cud("update Category set CategoryName='" + txtCategoryName.Text +
            "' where CategoryCode='" + txtCategoryCode.Text + "'");
            catt.Clear();
            db.CategoryTable("select * from Category");
            dgvCategory.DataSource = catt;
        }
    }

```

```

    }

private void btnAddStaff_Click(object sender, EventArgs e)
{
    db.cud("Insert into Staff(UserId, Username, Role, Contact, Password)
values('" + txtUserID.Text + "','" + txtuser_s_name.Text + "','" +
cmbDesignation.GetItemText(cmbDesignation.SelectedItem) + "','" + txtContact.Text + "','" +
txtPassword.Text + "'");
    st.Clear();
    db.staffTable("select * from Staff");
    dgvstaff.DataSource = st;
}

private void btnStaffEdit_Click(object sender, EventArgs e)
{
    DialogResult dialog = MessageBox.Show("Do you need to Edit", "Exit",
MessageBoxButtons.YesNo);
    if (dialog == DialogResult.Yes)
    {
        db.cud("update Staff set Username= '" + txtuser_s_name.Text +
"',Role='" + cmbDesignation.GetItemText(cmbDesignation.SelectedItem) + "', Contact=" +
txtContact.Text + ",Password='" + txtPassword.Text + "'where UserId='" + txtUserID.Text +
"'");
        st.Clear();
        db.staffTable("select * from Staff");
        dgvstaff.DataSource = st;
    }
}

private void btnStaffDelete_Click(object sender, EventArgs e)
{
    DialogResult dialog = MessageBox.Show("Do you need to delete", "Exit",
MessageBoxButtons.YesNo);
    if (dialog == DialogResult.Yes)
    {
        db.cud("delete from Staff where UserID = '" + txtUserID.Text + "'");
        st.Clear();
        db.staffTable("select * from Staff");
        dgvstaff.DataSource = st;
    }
}

private void btnStaffSearch_Click_1(object sender, EventArgs e)
{
    st.Clear();
    db.staffTable("Select*from Staff where Role ='" + cmbStaffsearch.Text + "'");
    dgvstaff.DataSource = st;
}

private void lblStaffRefresh_Click(object sender, EventArgs e)
{
    staffrefresh();
}
private void dgvstaff_CellClick(object sender, DataGridViewCellEventArgs e)
{
    var staff = dgvstaff.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;

```

```

        string s = staff.ToString();
        txtUserID.Text = dgvstaff.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtuser_s_name.Text = dgvstaff.Rows[e.RowIndex].Cells[1].Value.ToString();
        cmbDesignation.SelectedItem =
            dgvstaff.Rows[e.RowIndex].Cells[2].Value.ToString();
        txtContact.Text = dgvstaff.Rows[e.RowIndex].Cells[3].Value.ToString();
        txtPassword.Text = dgvstaff.Rows[e.RowIndex].Cells[4].Value.ToString();
    }

    private void E_MEd_Load(object sender, EventArgs e)
    {
        panelStaff.Visible = false;
        panelcustomer.Visible = false;
        panelCategory.Visible = false;
        panelProduct.Visible = false;
        panelOrder.Visible = false;
    }

    private void lblStaffReport_Click(object sender, EventArgs e)
    {
        StaffReport staff = new StaffReport();
        staff.Show();
    }

    private void lblProductReport_Click(object sender, EventArgs e)
    {
        Productreport prone = new Productreport();
        prone.Show();
    }

    private void dgvcatalogy_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        product = dgvcatalogy.CurrentRow.Cells[0].Value.ToString();
        stock = Convert.ToInt32(dgvcatalogy.CurrentRow.Cells[3].Value.ToString());
        uprice = Convert.ToInt32(dgvcatalogy.CurrentRow.Cells[2].Value.ToString());
        txtoderedpId.Text = dgvcatalogy.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtProname.Text = dgvcatalogy.Rows[e.RowIndex].Cells[1].Value.ToString();
        txtUnitPr.Text = dgvcatalogy.Rows[e.RowIndex].Cells[2].Value.ToString();
        flag = 1;
    }

    private void dgvproduct_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        var prod = dgvproduct.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
        string s = prod.ToString();
        txtProuctCode.Text = dgvproduct.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtProductName.Text = dgvproduct.Rows[e.RowIndex].Cells[1].Value.ToString();
        txtProductPrice.Text = dgvproduct.Rows[e.RowIndex].Cells[2].Value.ToString();
        txtQuantity.Text = dgvproduct.Rows[e.RowIndex].Cells[3].Value.ToString();
        txtProDec.Text = dgvproduct.Rows[e.RowIndex].Cells[4].Value.ToString();
        txtCategoryCode.Text = dgvproduct.Rows[e.RowIndex].Cells[5].Value.ToString();
    }

    private void dgvCategory_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        var demo = dgvCategory.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
        string s = demo.ToString();
        txtCategoryCode.Text = dgvCategory.Rows[e.RowIndex].Cells[0].Value.ToString();
    }

```

```

        txtCategoryName.Text =
dgvCategory.Rows[e.RowIndex].Cells[1].Value.ToString();
    }

    private void dgvCus_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        var demo = dgvCus.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;
        string s = demo.ToString();
        txtCustomerName.Text = dgvCus.Rows[e.RowIndex].Cells[0].Value.ToString();
        txtCustomerPhone.Text = dgvCus.Rows[e.RowIndex].Cells[1].Value.ToString();
        txtCustomerAddress.Text = dgvCus.Rows[e.RowIndex].Cells[2].Value.ToString();
    }

    private void lblAddOrder_Click(object sender, EventArgs e)
    {
        try
        {
            if (txtorderedquantity.Text == "")
            {
                MessageBox.Show("Enter The Quantity of Products");
            }
            else if (flag == 0)
            {
                MessageBox.Show("Select The Product");
            }
            else if (Convert.ToInt32(txtorderedquantity.Text) > stock)
            {
                MessageBox.Show("No Enough Stock Available");
            }
            else
            {
                num = Convert.ToInt32(txtoderedpId.Text);
                qty = Convert.ToInt32(txtorderedquantity.Text);
                cname = txtOrderedCusto.Text;
                totprice = qty * uprice;
                tab.Rows.Add(num, product, qty, uprice, totprice);
                dgvorder.DataSource = tab;
                flag = 0;

                updateproduct();
            }
        }
        catch
        {
            MessageBox.Show("Please check the order again");
        }
    }

    private void comboBox2_SelectedIndexChanged(object sender, EventArgs e)
    {
        pt.Clear();
        db.ProdctTable("Select*from Product where CategoryCode ='" +
cmboderedCat.Text + "'");
        dgvcatogory.DataSource = pt;}}

```

References

- [1] D. Lee, "Pharmacy2U," [Online]. Available: <https://www.pharmacy2u.co.uk/>.
- [2] Selikta, "unionchemistspharmacy," [Online]. Available: <https://www.unionchemistspharmacy.lk/about-us/>.
- [3] apollohospitals, "apollohospitals," [Online]. Available: <https://www.apollohospitals.com>.
- [4] A. Solutions, "healthguard," [Online]. Available: <https://www.healthguard.lk/>.

